



Diplomski studij

**Informacijska i komunikacijska
tehnologija**

Računarstvo

Telekomunikacije i informatika

**Obradba informacija
Računalno inženjerstvo**

Internet stvari

Smart parking

Projekt

Elena Neseck

Petar Kapec

Karlo Raštegorgac

Matan Budimir

Sven Đukić

Ak.g. 2025./2026.

Sadržaj

- 1. Uvod3**
- 2. Opis rješenja3**
- 3. IoT platforma13**
- 4. Korisničke aplikacije18**

1. Uvod

SmartParking je IoT sustav namijenjen detekciji i prikazu zauzetosti parkirnih mjesta u stvarnom vremenu. Sustav je razvijen s ciljem primjene tehnologija Interneta stvari na konkretan problem u području pametnih gradova i prometa. Problem pronalaska slobodnog parkirnog mjesta čest je u urbanim sredinama, osobito na lokacijama s velikom frekvencijom vozila, kao što su fakulteti, trgovački centri, poslovne zone, javne garaže i gradska parkirališta. Vozači često provode dodatno vrijeme kružeći oko parkirališta u potrazi za slobodnim mjestom, što povećava prometno opterećenje, potrošnju goriva i emisiju štetnih plinova.

Cilj SmartParking sustava je korisnicima omogućiti jednostavan i brz pregled dostupnih parkirnih mjesta putem web aplikacije. Umjesto fizičkog obilaska parkirališta, korisnik može otvoriti aplikaciju, odabrati parking na karti i vidjeti trenutno stanje zauzetosti. Na taj način sustav doprinosi učinkovitijem korištenju parkirnog prostora, smanjenju vremena traženja parkinga i boljem upravljanju prometom u okolini parkirališta. Osim koristi za krajnje korisnike, sustav može biti koristan i upraviteljima parkirališta jer omogućuje praćenje stanja popunjenosti i kasniju analizu korištenja parkirnih kapaciteta.

U sklopu rješenja jedno parkirno mjesto opremljeno je fizičkim senzorskim čvorom za detekciju prisutnosti vozila, dok se ostala parkirna mjesta simuliraju programski. Fizički dio sustava temelji se na ESP32 mikrokontroleru i ultrazvučnom senzoru HC-SR04. Ultrazvučni senzor mjeri udaljenost do objekta ispred sebe i na temelju te udaljenosti može se zaključiti nalazi li se vozilo na parkirnom mjestu. Ako je udaljenost manja od zadanog praga, sustav mjesto označava kao zauzeto, a ako je udaljenost veća, mjesto se označava kao slobodno. Dobiveni podatak šalje se dalje kroz sustav prema mrežnom prilazu, poslužitelju i korisničkoj aplikaciji.

Kao aktuator koristi se LED dioda koja lokalno prikazuje stanje parkirnog mjesta. Zelena LED dioda označava da je mjesto slobodno. Takav lokalni prikaz koristan je za testiranje i demonstraciju rada fizičkog senzorskog čvora jer se stanje može provjeriti i bez pristupa web aplikaciji. LED dioda ujedno pokazuje da sustav ne služi samo za udaljeni prikaz podataka, nego može pružiti i neposrednu povratnu informaciju u fizičkom prostoru.

Budući da u projektu nije bilo predviđeno opremanje svakog parkirnog mjesta stvarnim senzorom, ostala parkirna mjesta prikazana su pomoću simulacije. Simulacija se ponaša kao skup virtualnih senzora koji periodički generiraju promjene stanja parkirnih mjesta. Na taj način moguće je prikazati rad sustava na više parkirnih mjesta i više parkirališta, iako je fizički implementirano samo jedno mjesto. Simulirani podaci se obrađuju na isti način kao i podaci stvarnog senzora: šalju se prema poslužitelju, spremaju u bazu podataka i prikazuju u korisničkoj aplikaciji.

Sustav omogućuje više slučajeva uporabe. Prvi slučaj uporabe je pregled slobodnih parkirnih mjesta putem web aplikacije. Korisnik pristupa aplikaciji, na karti odabire željeni parking i dobiva prikaz ukupnog broja mjesta, broja slobodnih i zauzetih mjesta te postotka popunjenosti. Uz to se prikazuje i status svakog pojedinog parkirnog mjesta. Drugi slučaj uporabe je automatsko ažuriranje zauzetosti. Kada senzor ili simulacija detektiraju promjenu stanja, promjena se šalje poslužitelju, sprema u bazu podataka i

automatski prikazuje u korisničkom sučelju bez potrebe za ručnim osvježavanjem stranice. Treći mogući slučaj uporabe je korištenje sustava od strane administratora ili upravitelja parkinga, koji može pratiti opterećenje parkirališta i planirati bolje upravljanje dostupnim prostorom.

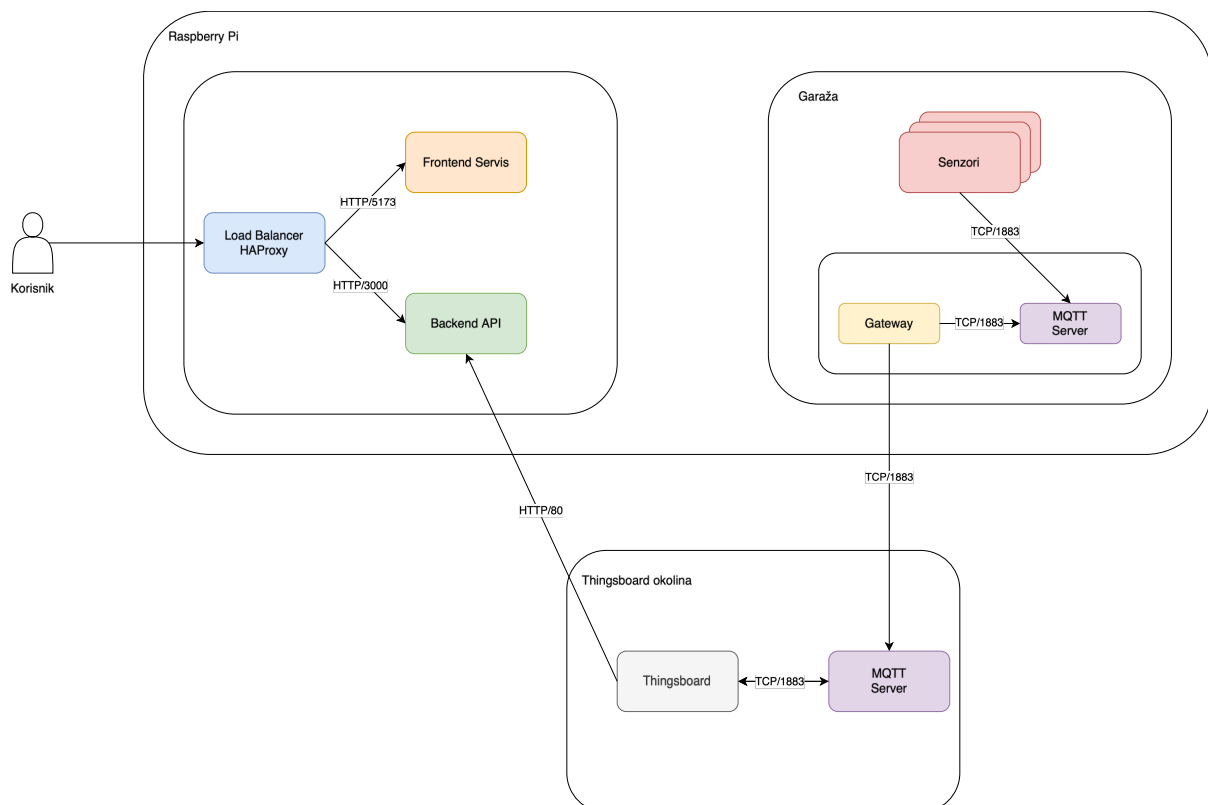
Slična rješenja već postoje u području pametnih gradova. Primjer je Santander Smart Parking, aplikacija koja koristi podatke iz senzorskog sustava za prikaz dostupnosti parkirnih mjesta u stvarnom vremenu. Širi primjer primjene IoT tehnologije u gradu je projekt SmartSantander, u kojem su senzori korišteni za različite gradske usluge, uključujući promet i navođenje vozača prema slobodnim parkirnim mjestima. U Barceloni se kao primjer može navesti rješenje Parclic, koje korisnicima omogućuje pronalazak i rezervaciju parkirnih mjesta, dok se u Amsterdamu koristi EasyPark, aplikacija za pronalazak i upravljanje parkiranjem. SmartParking projekt slijedi istu osnovnu ideju, ali je prilagođen studentskom projektnom okruženju i kombinira fizički senzor, simulaciju, IoT platformu, backend sustav i web aplikaciju.

2. Opis rješenja

Sustav SmartParking osmišljen je kao višeslojno IoT rješenje za detekciju i prikaz zauzetosti parkirnih mjesta u stvarnom vremenu. Arhitektura sustava povezuje fizičke senzore, mrežni prilaz, MQTT komunikaciju, IoT platformu ThingsBoard, aplikacijski poslužitelj, bazu podataka, simulacijski modul i korisničku web aplikaciju. Takav način organizacije omogućuje jasno razdvajanje odgovornosti pojedinih komponenti i lakše proširenje sustava u budućnosti.

Osnovna ideja sustava je da se stanje parkirnog mjesta očitava pomoću senzora ili simulacije, zatim se promjena šalje prema poslužiteljskom dijelu sustava, sprema u bazu podataka i odmah prikazuje korisniku putem web aplikacije. Korisnik ne mora ručno osvježavati stranicu jer se promjene šalju u stvarnom vremenu korištenjem WebSocket veze.

Arhitektura sustava može se podijeliti na nekoliko glavnih cjelina: fizički IoT dio u garaži, lokalni mrežni prilaz, poslužiteljski aplikacijski dio, IoT platformu ThingsBoard, simulacijski dio i korisničku aplikaciju. Fizički dio sustava koristi ESP32 mikrokontroler, ultrazvučni senzor HC-SR04 i LED diodu. Simulacijski dio koristi se za predstavljanje dodatnih parkirnih mjesta koja nisu pokrivena fizičkim senzorima. Poslužiteljski dio aplikacije implementiran je pomoću Node.js i Express okruženja, dok je korisnička aplikacija izrađena u Reactu i prikazuje parkirališta na OpenStreetMap karti.



2.1. Pregled arhitekture

Arhitektura sustava prikazana je na skici rješenja. Na lijevoj strani nalazi se aplikacijski dio sustava kojem korisnik pristupa putem web preglednika. Korisnički zahtjevi prvo dolaze do

HAProxy load balancera, koji ima ulogu ulazne točke u aplikacijski sustav. HAProxy na temelju vrste zahtjeva odlučuje treba li zahtjev proslijediti frontend servisu ili backend API servisu.

Frontend servis poslužuje React web aplikaciju. Korisnik putem aplikacije može vidjeti kartu, odabrati parkiralište i pregledati stanje parkirnih mjesta. Backend API servis obrađuje podatke, dohvaća stanje iz baze, prima promjene iz simulacije ili senzorskog dijela sustava te šalje promjene web aplikaciji u stvarnom vremenu.

Na desnoj strani arhitekture prikazan je fizički dio sustava koji se nalazi u garaži. U garaži se nalaze senzori koji detektiraju prisutnost vozila. Ti senzori komuniciraju putem MQTT protokola s MQTT serverom. Raspberry Pi ima ulogu mrežnog prilaza, odnosno posrednika između fizičkog senzorskog dijela i ostatka sustava. Gateway komponenta prosljeđuje podatke prema vanjskom MQTT serveru i ThingsBoard platformi.

ThingsBoard server koristi se kao IoT platforma za prihvatanje, nadzor i prikaz telemetrijskih podataka. U arhitekturi je prikazana i komunikacija prema ThingsBoardu putem MQTT protokola na portu 1883. Time se omogućuje da podaci s fizičkih senzora budu dostupni na IoT platformi, gdje se mogu prikazivati pomoću dashboarda i analizirati kao telemetrijski podaci.

2.2. Glavne komponente sustava

Glavne komponente sustava su:

- IoT uređaj s ESP32 mikrokontrolerom
- ultrazvučni senzor HC-SR04
- LED dioda kao aktuator
- Raspberry Pi kao mrežni prilaz
- MQTT broker
- gateway servis
- ThingsBoard IoT platforma
- backend API servis
- SQLite baza podataka
- frontend web aplikacija
- HAProxy load balancer
- simulacijski servis
- ngrok servis za javni pristup aplikaciji

Svaka komponenta ima jasno definiranu ulogu u sustavu. Fizički senzorski dio zadužen je za prikupljanje podataka iz stvarnog okruženja. MQTT broker omogućuje razmjenu poruka između IoT komponenti. Backend obrađuje podatke i upravlja bazom. Frontend prikazuje podatke korisniku. HAProxy omogućuje pristup cijeloj aplikaciji preko jedne ulazne adrese. Docker Compose povezuje sve softverske komponente u jedinstveno pokretno okruženje.

2.3. Fizički IoT uređaj

Fizički IoT uređaj predstavlja jedno stvarno parkirno mjesto. Sastoji se od ESP32 mikrokontrolera, ultrazvučnog senzora HC-SR04 i LED dioda. ESP32 je pogodan za IoT projekte jer ima ugrađenu Wi-Fi podršku, dovoljan broj GPIO pinova i mogućnost povezivanja s MQTT brokerom.

Ultrazvučni senzor HC-SR04 koristi se za mjerenje udaljenosti između senzora i objekta ispred njega. U kontekstu parkirnog mjesta, objekt predstavlja vozilo. Ako je izmjerena udaljenost manja od unaprijed definiranog praga, sustav zaključuje da je parkirno mjesto zauzeto. Ako je udaljenost veća od praga, mjesto se smatra slobodnim.

LED dioda koristi se kao lokalni aktuator. Upaljena zelena LED dioda označava slobodno mjesto, dok ugašena dioda označava zauzeto mjesto. Time se omogućuje jednostavna lokalna vizualizacija stanja mjesta i bez korištenja web aplikacije. Ovakav prikaz koristan je tijekom testiranja fizičkog uređaja jer se odmah može vidjeti prepoznaje li sustav ispravno prisutnost vozila.

ESP32 nakon obrade mjerenja formira poruku koja sadrži identifikator parkirnog mjesta, stanje zauzetosti i izvor podatka. Poruka se zatim šalje putem MQTT protokola prema mrežnom prilazu, odnosno Raspberry Pi uređaju.

Logika rada uređaja temelji se na ciklusu koji se ponavlja u stalnim vremenskim razmacima. U svakom ciklusu mikrokontroler najprije izvodi mjerenje udaljenosti, zatim na temelju izmjerene vrijednosti određuje stanje parkirnog mjesta, ažurira lokalni prikaz pomoću LED dioda i na kraju, ako je došlo do promjene stanja, šalje poruku putem MQTT protokola.

Mjerenje udaljenosti izvodi se tako da se na okidački pin senzora pošalje kratki impuls, nakon čega senzor emitira ultrazvučni signal i mjeri vrijeme do povratka odjeka. Iz izmjerene vremena trajanja odjeka izračunava se udaljenost na temelju brzine zvuka u zraku. Dobivena udaljenost u centimetrima zatim se uspoređuje s unaprijed definiranim pragom. Ako je udaljenost veća od nule i manja od praga, smatra se da se ispred senzora nalazi vozilo te se mjesto označava kao zauzeto; u suprotnom se mjesto smatra slobodnim.

Na temelju utvrđenog stanja mikrokontroler upravlja LED diodama kao lokalnim aktuatorima. Promjena stanja diode izvodi se samo kada se stvarno stanje razlikuje od trenutno prikazanog, čime se izbjegava nepotrebno ponavljanje istih radnji i ispisivanje istih poruka.

Slanje podataka prema mrežnom prilazu izvedeno je tako da se MQTT poruka ne šalje u svakom ciklusu, nego samo pri prvom očitavanju i pri svakoj promjeni stanja zauzetosti. Time se znatno smanjuje mrežni promet i opterećenje brokera, budući da se podaci prenose tek kada dođe do stvarne promjene. Poruka se formira u JSON formatu i sadrži identifikator parkinga, identifikator parkirnog mjesta, stanje zauzetosti, oznaku je li uređaj online, izvor podatka i vremensku oznaku. Prije slanja uređaj provjerava postojanje veze s MQTT brokerom te se po potrebi automatski ponovno spaja.

Postojeća logika izvodi zaključak na temelju jednog mjerenja po ciklusu i fiksnog praga. Upravo je to mjesto na kojem bi naprednija, odnosno na umjetnoj inteligenciji zasnovana obrada mogla donijeti poboljšanja. Ultrazvučni senzori podložni su povremenim pogrešnim očitanjima zbog odbijanja signala, šuma ili graničnih udaljenosti, pa bi se filtriranjem više uzastopnih mjerenja (primjerice medijanom ili pomičnim prosjekom) i uvođenjem histereze oko praga mogle ukloniti lažne promjene stanja. Naprednija obrada mogla bi i prepoznavati obrasce zauzetosti tijekom vremena te predviđati dostupnost mjesta, no u ovoj je inačici sustava namjerno zadržana jednostavna i predvidljiva logika radi lakšeg testiranja i razumijevanja rada uređaja.

2.4. Raspberry Pi kao mrežni prilaz

Raspberry Pi u sustavu ima ulogu mrežnog prilaza. Mrežni prilaz služi kao posrednik između fizičkih IoT uređaja i ostatka sustava. U stvarnim IoT sustavima mrežni prilaz često se koristi

zato što senzorski uređaji mogu imati ograničene mogućnosti obrade, ograničenu memoriju ili se nalaziti u lokalnoj mreži koja nije izravno dostupna iz oblaka.

U ovom sustavu Raspberry Pi može primiti MQTT poruke od ESP32 senzora i prosljeđivati ih prema drugim komponentama sustava. Na Raspberry Pi uređaju može biti pokrenut lokalni Mosquitto MQTT broker ili gateway komponenta koja se spaja na broker i prosljeđuje poruke prema ThingsBoardu ili internom MQTT brokeru u Docker okruženju.

Prednost korištenja Raspberry Pi uređaja je u tome što on omogućuje fleksibilnije upravljanje komunikacijom. Ako se kasnije doda više fizičkih senzora, oni se mogu povezivati na isti mrežni prilaz. Raspberry Pi zatim može objedinjavati poruke, filtrirati ih, provjeravati ispravnost podataka i prosljeđivati ih dalje prema aplikacijskom dijelu sustava.

2.5. MQTT komunikacija

MQTT je komunikacijski protokol često korišten u IoT sustavima. Temelji se na publish-subscribe modelu. To znači da uređaji ne šalju poruke izravno jedan drugome, nego poruke objavljuju na određene teme, a drugi uređaji ili servisi se pretplaćuju na te teme.

U SmartParking sustavu MQTT se koristi za komunikaciju između senzorskog dijela, gatewaya, simulacije, internog MQTT brokera i ThingsBoard platforme. MQTT broker prima poruke od publisher-a i prosljeđuje ih svim subscriberima koji su pretplaćeni na odgovarajuće teme.

Primjer MQTT poruke može sadržavati podatke poput identifikatora parkinga, identifikatora parkirnog mjesta, stanja zauzetosti, informacije je li senzor online i izvora podatka. Takav format omogućuje backendu da zna koje mjesto treba ažurirati i treba li promjenu prikazati kao podatak dobiven sa senzora ili iz simulacije.

U Docker Compose konfiguraciji koristi se Eclipse Mosquitto kao MQTT broker. Mosquitto je lagan i često korišten MQTT broker prikladan za IoT projekte. Konfiguracijske datoteke brokera nalaze se u direktoriju `docker/mqtt`, a u Docker container se montiraju pomoću volume mehanizma.

2.6. ThingsBoard IoT platforma

ThingsBoard se koristi kao IoT platforma za nadzor i prikaz telemetrijskih podataka. Uloga ThingsBoarda nije ista kao uloga backend aplikacije. Backend je zadužen za podatke koji se prikazuju u korisničkoj web aplikaciji, dok ThingsBoard služi za IoT nadzor, obradu telemetrije i izradu dashboarda za uređaje.

U sustavu se podaci s fizičkog senzora mogu prosljeđivati prema ThingsBoardu putem MQTT protokola. ThingsBoard prima podatke na MQTT brokeru i povezuje ih s registriranim uređajem. Svaki uređaj u ThingsBoardu ima svoj pristupni token, pomoću kojeg se autentificira pri slanju telemetrije.

Na ThingsBoard dashboardu mogu se prikazivati vrijednosti kao što su udaljenost izmjerena senzorom, status zauzetosti parkirnog mjesta, vrijeme zadnjeg očitavanja i status uređaja. Time se omogućuje tehnički nadzor IoT uređaja neovisno o korisničkoj web aplikaciji.

2.7. Backend API servis

Backend API servis implementiran je pomoću Node.js i Express radnog okvira. Njegova glavna uloga je upravljanje podacima o parkiralištima i parkirnim mjestima. Backend prima zahtjeve od frontend aplikacije, dohvaća podatke iz SQLite baze, prima promjene stanja iz simulacije ili senzorskog dijela sustava i šalje ažuriranja korisnicima putem WebSocket veze.

Backend izlaže REST API endpoint za dohvat svih parkinga i njihovih mjesta. Kada korisnik otvori web aplikaciju, frontend šalje zahtjev prema backendu i dohvaća početno stanje sustava. To početno stanje uključuje lokacije parkinga, nazive parkinga, broj mjesta i status svakog pojedinog mjesta.

Osim REST API-ja, backend koristi i WebSocket poslužitelj. WebSocket omogućuje dvosmjernu komunikaciju između backenda i web aplikacije. U ovom projektu najvažnija je mogućnost da backend sam pošalje promjenu frontendu čim se dogodi promjena stanja parkirnog mjesta. Time se izbjegava potreba da frontend svake sekunde šalje nove REST zahtjeve i provjerava je li se nešto promijenilo.

Kada backend primi promjenu stanja parkirnog mjesta, on najprije validira podatke, zatim ažurira zapis u SQLite bazi i nakon toga šalje WebSocket poruku svim spojenim klijentima. Frontend prima tu poruku i ažurira prikaz bez osvježavanja stranice.

2.8. SQLite baza podataka

Za pohranu podataka koristi se SQLite baza podataka. SQLite je prikladan za ovaj projekt jer je jednostavan za korištenje, ne zahtijeva poseban database server i svi se podaci nalaze u jednoj .db datoteci. U Docker verziji projekta baza se nalazi u direktoriju `docker/db/smartparking.db` i montira se u backend container.

Baza sadrži dvije glavne tablice: `parking` i `parking_spot`. Tablica `parking` sadrži osnovne informacije o parkiralištima, kao što su identifikator, naziv, geografska širina i geografska dužina. Te se koordinate koriste za prikaz markera na OpenStreetMap karti.

Tablica `parking_spot` sadrži podatke o pojedinim parkirnim mjestima. Svako mjesto povezano je s pripadajućim parkingom pomoću stranog ključa. U tablici se čuvaju oznaka mjesta, stanje zauzetosti, informacija je li mjesto online, izvor podatka i vrijeme zadnjeg ažuriranja.

Korištenje baze omogućuje da se stanje sustava ne izgubi nakon osvježavanja web stranice ili ponovnog pokretanja aplikacije. Ako simulacija ili senzor promijene stanje mjesta, backend promjenu zapisuje u bazu. Kada korisnik kasnije ponovno otvori aplikaciju, dobiva zadnje spremljeno stanje.

2.9. Simulacijski modul

Budući da sustav nema fizički senzor za svako parkirno mjesto, koristi se simulacijski modul. Simulacija predstavlja virtualne senzore za parkirna mjesta koja nisu fizički opremljena senzorima. Time se omogućuje prikaz rada sustava na većem broju parkinga i parkirnih mjesta, bez potrebe za većim brojem fizičkih uređaja.

Simulacijski servis pokreće se kao zaseban Docker container. Pri pokretanju dohvaća podatke o postojećim parkinzima iz backenda. Nakon toga periodički generira promjene stanja za pojedina parkirna mjesta. U logovima simulacije može se vidjeti da simulacija ispisuje za koji parking i koje mjesto je generirana promjena.

Simulacija šalje podatke prema backendu preko HTTP zahtjeva i/ili MQTT komunikacije, ovisno o implementaciji servisa. U Docker Compose konfiguraciji simulacija dobiva potrebne konfiguracijske podatke preko environment varijabli: `MQTT_BROKER`, `MQTT_USERNAME`, `MQTT_PASSWORD` i `BACKEND_URL`.

Važno je da `BACKEND_URL` u Docker okruženju ne bude postavljen na `localhost`, nego na naziv servisa `backend`. U Docker mreži `localhost` označava sam container, a ne cijeli sustav. Zato se backend iz simulacijskog containera dohvaća putem adrese `http://backend:3000`.

Simulacija je korisna za razvoj i demonstraciju jer omogućuje da se korisničko sučelje stalno mijenja i prikazuje realističan scenarij u kojem se mjesta oslobađaju i zauzimaju.

2.10. Frontend web aplikacija

Frontend aplikacija razvijena je pomoću Reacta i Vite razvojnog okruženja. Za prikaz karte koristi se Leaflet biblioteka i OpenStreetMap. Aplikacija korisniku prikazuje kartu s markerima koji označavaju lokacije parkirališta. Klikom na marker korisnik odabire parking i dobiva detaljan prikaz stanja parkirnih mjesta.

U detaljnom prikazu prikazuju se osnovni pokazatelji: ukupan broj mjesta, broj slobodnih mjesta, broj zauzetih mjesta i postotak popunjenosti. Ispod statistike prikazuju se pojedina parkirna mjesta. Slobodna mjesta prikazuju se zelenom bojom, zauzeta crvenom, a offline mjesta sivom bojom.

Frontend pri pokretanju najprije dohvaća početno stanje parkinga preko REST API-ja. Nakon toga uspostavlja WebSocket vezu s backendom. Kada backend pošalje poruku o promjeni stanja mjesta, frontend ažurira lokalno stanje aplikacije i odmah prikazuje promjenu korisniku.

Važno je da frontend u Docker verziji ne koristi ručno upisanu IP adresu backenda, nego zahtjeve šalje preko iste domene s koje je aplikacija otvorena. Time se omogućuje da aplikacija radi i na `localhost`, i na IP adresi računala, i preko ngrok tunela, bez potrebe za mijenjanjem koda.

2.11. HAProxy load balancer

HAProxy je u sustavu postavljen kao jedina ulazna točka za korisnički promet. Korisnik aplikaciji pristupa preko adrese `http://localhost`, odnosno preko porta 80. HAProxy zatim odlučuje kamo će proslijediti zahtjev.

Ako zahtjev ide prema frontend dijelu aplikacije, HAProxy ga prosljeđuje frontend servisu. Ako zahtjev ide prema API putanji, HAProxy ga prosljeđuje backend servisu. Ovakav pristup

omogućuje da korisnik ne mora znati na kojem portu se nalazi frontend, a na kojem backend. Svi zahtjevi idu preko jedne adrese.

Prednost korištenja HAProxyja je i u tome što se kasnije može lakše proširiti sustav. Na primjer, moguće je dodati više instanci backend servisa ili promijeniti interne portove bez promjene korisničke adrese aplikacije. HAProxy također omogućuje ispravno prosljeđivanje WebSocket veza, što je važno za real-time ažuriranje prikaza.

2.12. ngrok servis

U Docker Compose konfiguraciji nalazi se i ngrok servis. Ngrok omogućuje izradu javnog tunela prema lokalnoj aplikaciji. To znači da aplikacija koja se pokreće lokalno na računalu može dobiti javno dostupnu HTTPS adresu. Takav pristup koristan je za demonstraciju projekta ili testiranje s drugim uređajima koji nisu u istoj lokalnoj mreži.

Ngrok servis ovisi o HAProxy servisu jer javni tunel prosljeđuje promet prema HAProxyju. Autentikacija se obavlja pomoću `NGROK_AUTHTOKEN` varijable okruženja. Ako ta varijabla nije postavljena, ngrok servis se neće ispravno pokrenuti, ali to ne sprječava lokalni rad aplikacije. Lokalno je aplikacija i dalje dostupna preko `http://localhost`.

2.13. Docker Compose

Cijeli aplikacijski dio sustava pokreće se pomoću Docker Composea. Docker Compose omogućuje da se više povezanih servisa definira u jednoj YAML datoteci i pokrene jednom naredbom. U ovom projektu definirani su servisi: `mqtt`, `backend`, `frontend`, `haproxy`, `ngrok`, `simulation` i `gateway`.

Servis `mqtt` koristi image `eclipse-mosquitto:2`. To je lokalni MQTT broker. Servis `backend` koristi image `matanbudimir/intstv:backend` i montira SQLite bazu iz lokalne datoteke. Servis `frontend` koristi image `matanbudimir/intstv:frontend`. Servis `haproxy` koristi image `haproxy:alpine` i izlaže port 80 prema korisniku. Servis `simulation` koristi image `matanbudimir/intstv:simulation`, a servis `gateway` koristi image `matanbudimir/intstv:gateway`.

Docker Compose također definira ovisnosti između servisa. Primjerice, simulacija ne kreće odmah, nego čeka da MQTT servis bude pokrenut i da backend prođe healthcheck. Healthcheck za backend provjerava je li backend dostupan na portu 3000. Time se sprječava situacija u kojoj simulacija pokuša dohvatiti podatke iz backenda prije nego što je backend spreman.

Korištenje Docker Composea pojednostavljuje pokretanje sustava jer korisnik ne mora ručno pokretati svaki servis. Dovoljno je pokrenuti naredbu `docker compose up -d`, nakon čega se svi servisi pokreću u ispravnom redoslijedu.

2.14. Tok podataka od senzora do korisnika

Cjeloviti tok podataka počinje na fizičkom senzoru. HC-SR04 mjeri udaljenost i ESP32 na temelju te vrijednosti određuje je li mjesto zauzeto. Nakon toga ESP32 šalje MQTT poruku

prema Raspberry Pi mrežnom prilazu. Gateway preuzima poruku i prosljeđuje je prema MQTT brokeru i ThingsBoard platformi.

Ako se podatak koristi za prikaz u web aplikaciji, backend prima promjenu, ažurira bazu podataka i šalje WebSocket obavijest frontendu. Frontend zatim mijenja prikaz parkirnog mjesta. Korisnik vidi promjenu u sučelju bez ručnog osvježavanja stranice.

Kod simuliranih mjesta tok je kraći. Simulacijski servis generira promjenu i šalje je backendu. Backend je obrađuje na isti način kao i promjenu dobivenu od stvarnog senzora: ažurira bazu i obavještava frontend. Time se postiže jedinstvena obrada podataka, neovisno o tome dolaze li iz fizičkog senzora ili simulacije.

2.15. Prednosti ovakve arhitekture

Ovakva arhitektura ima nekoliko prednosti. Prva prednost je modularnost. Svaka komponenta sustava ima jasno definiranu ulogu. Frontend je zadužen za prikaz, backend za obradu podataka, baza za pohranu, MQTT za komunikaciju IoT komponenti, a ThingsBoard za IoT nadzor.

Druga prednost je skalabilnost. Ako se u budućnosti doda više senzora, oni se mogu povezati na isti MQTT sustav. Ako se doda više parkinga, dovoljno ih je dodati u bazu podataka, a simulacija i web aplikacija mogu ih prikazivati bez većih promjena u arhitekturi.

Treća prednost je jednostavno pokretanje. Docker Compose omogućuje da svi servisi rade u izoliranim kontejnerima. Time se smanjuju problemi vezani uz različite verzije Node.js-a, lokalne konfiguracije i instalaciju ovisnosti.

Četvrta prednost je real-time prikaz. Korištenjem WebSocket veze korisnik odmah vidi promjene stanja. To je važno za SmartParking sustav jer informacije o slobodnim mjestima brzo gube vrijednost ako nisu ažurne.

Peta prednost je kombiniranje stvarnih i simuliranih podataka. Sustav se može demonstrirati i testirati s više parkirnih mjesta čak i ako postoji samo jedan fizički senzor. Time se olakšava razvoj i smanjuju troškovi hardvera.

3. IoT platforma

U sklopu SmartParking sustava kao IoT platforma koristi se **ThingsBoard**. ThingsBoard je open-source IoT platforma namijenjena prikupljanju, obradi, vizualizaciji i upravljanju podacima s IoT uređaja. U ovom projektu platforma se koristi za prihvatanje telemetrijskih podataka sa stvarnog senzorskog uređaja i za prikaz tih podataka na dashboardu.

ThingsBoard podržava povezivanje IoT uređaja korištenjem više standardnih komunikacijskih protokola, među kojima su MQTT, HTTP i CoAP. Za SmartParking sustav najvažniji je MQTT protokol jer je prikladan za IoT uređaje, ima mali komunikacijski overhead i često se koristi za slanje kratkih telemetrijskih poruka sa senzora. U našem slučaju podaci o zauzetosti parkirnog mjesta šalju se prema ThingsBoard platformi putem MQTT komunikacije.

Osnovna uloga ThingsBoard platforme u ovom projektu je nadzor stvarnog IoT uređaja. ESP32 s ultrazvučnim senzorom mjeri prisutnost vozila na parkirnom mjestu, a dobiveni podaci se šalju prema platformi. Na ThingsBoardu se zatim mogu prikazivati vrijednosti kao što su trenutno stanje parkirnog mjesta, izmjerena udaljenost, vrijeme zadnjeg očitavanja i status uređaja. Time se omogućuje tehnički pregled rada fizičkog senzora neovisno o korisničkoj web aplikaciji.

ThingsBoard omogućuje kreiranje uređaja, definiranje pristupnih tokena, primanje telemetrije, spremanje vremenskih podataka, izradu dashboarda i prikaz podataka u stvarnom vremenu. Dashboard može sadržavati različite widgete, primjerice indikator zauzetosti mjesta, graf izmjerenih vrijednosti kroz vrijeme ili prikaz zadnje primljene poruke. Takav prikaz koristan je za provjeru rada senzora i komunikacije između fizičkog uređaja, gatewaya i platforme.

Platforma se može koristiti u različitim oblicima. ThingsBoard Community Edition je besplatna open-source verzija koja se može samostalno instalirati na vlastiti poslužitelj. Osim toga, dostupne su i komercijalne opcije, poput ThingsBoard Cloud i Professional Edition, koje nude dodatne mogućnosti i hosting u oblaku. Za potrebe studentskog projekta dovoljne su osnovne mogućnosti platforme, odnosno prihvatanje telemetrije, prikaz podataka i jednostavan dashboard.

U sklopu SmartParking sustava kao IoT platforma koristi se ThingsBoard (verzija 3.9.1). ThingsBoard je open-source IoT platforma namijenjena prikupljanju, obradi, vizualizaciji i upravljanju podacima s IoT uređaja. U ovom projektu platforma se koristi za prihvatanje telemetrijskih podataka sa stvarnog senzorskog uređaja i simulacije, te za obradu i prikaz tih podataka na korisničkoj nadzornoj ploči (eng. Dashboard).

ThingsBoard podržava povezivanje IoT uređaja korištenjem više standardnih komunikacijskih protokola, među kojima su MQTT, HTTP i CoAP. Za SmartParking sustav najvažniji je MQTT protokol jer je prikladan za IoT uređaje, ima mali komunikacijski overhead i često se koristi za slanje kratkih telemetrijskih poruka sa senzora. U našem slučaju, podaci o zauzetosti parkirnog mjesta šalju se prema ThingsBoard platformi putem MQTT komunikacije.

Osnovna uloga ThingsBoard platforme u ovom projektu je nadzor stvarnog IoT uređaja i simuliranih podataka na centraliziranom mjestu. Sustav prikuplja podatke s ESP32 mikrokontrolera (koji očitava HC-SR04 ultrazvučni senzor) i simulacijskog modula, a platforma ih integrira u jedinstven sustav prikaza.

3.1. Korištene mogućnosti IoT platforme

Kako bi se ostvarila funkcionalnost praćenja parkirališta u stvarnom vremenu, u projektu su korištene sljedeće ključne komponente ThingsBoard platforme:

Upravljanje uređajima (Device Management): Fizički senzor registriran je kao jedinstveni uređaj unutar platforme (ESP32_parking_senzor), s dodijeljenim pristupnim tokenom za sigurnu MQTT autentikaciju.

Telemetrija (Telemetry): Kontinuirano prikupljanje i vremensko označavanje (eng. timestamping) podataka o stanju parkirnih mjesta (slobodno/zauzeto), udaljenosti senzora i aktivnoj garaži.

Lanac pravila (Rule Engine): Sustav vizualnog programiranja temeljen na čvorovima (eng. nodes) koji služi za filtriranje, transformaciju i usmjeravanje dolaznih poruka prije nego što se one spremaju u bazu podataka.

Nadzorna ploča (Dashboard): Vizualno sučelje koje u stvarnom vremenu prikazuje obrađene podatke pomoću dinamičkih HTML/CSS kartica.

3.2. Konfiguracija resursa (senzori i aktuatori)

Fizički senzor i simulacijski backend platformi pristupaju kao isti entitet, stoga je najveći izazov prilikom konfiguracije resursa bio ispravno razdvojiti podatke koji pristižu s fizičkog senzora (mjesto A1) od podataka koji pristižu iz simulacije (ostala mjesta i garaže).

Podaci na platformu stižu u JSON formatu. Kako bi se izbjeglo preklapanje podataka na nadzornoj ploči, unutar Rule Engine sučelja ubačen je prilagođeni Script čvor za transformaciju (eng. Transformation Script). Skripta presreće svaku dolaznu MQTT poruku, analizira postoji li u njoj specifičan identifikator garaže (parking_id) i mjesta te stvara univerzalne telemetrijske ključeve (tip_mjesta, zadnja_garaza, zadnje_mjesto, zadnje_stanje) neophodne za dinamički prikaz.

Kod korišten za transformaciju i usmjeravanje telemetrije prikazan je u nastavku:

JavaScript

```
var spot = msg.spotId;
```

```
var occ = msg.occupied;
```

```
// 1. ID-a garaže (podržava više formata)
```

```
var parkId = msg.parking_id;

if (parkId == null) {

    parkId = msg.parkingId;

}

if (parkId == null) {

    parkId = "parking_demo"; // Zadana garaža za fizički senzor

}

// 2. Kreiranje univerzalnih ključeva za dinamičku karticu

msg.zadnja_garaza = parkId;

msg.zadnje_stanje = occ;

var trenutnoMjesto = (spot == null) ? "A1" : spot;

msg.zadnje_mjesto = trenutnoMjesto;

// 3. Klasifikacija izvora (Fizički senzor/Simulacija)

if (spot == "A1" || spot == null) {

    msg.spotId = "A1";

    msg.source = "sensor";

    msg.tip_mjesta = "Fizičko mjesto";

} else {

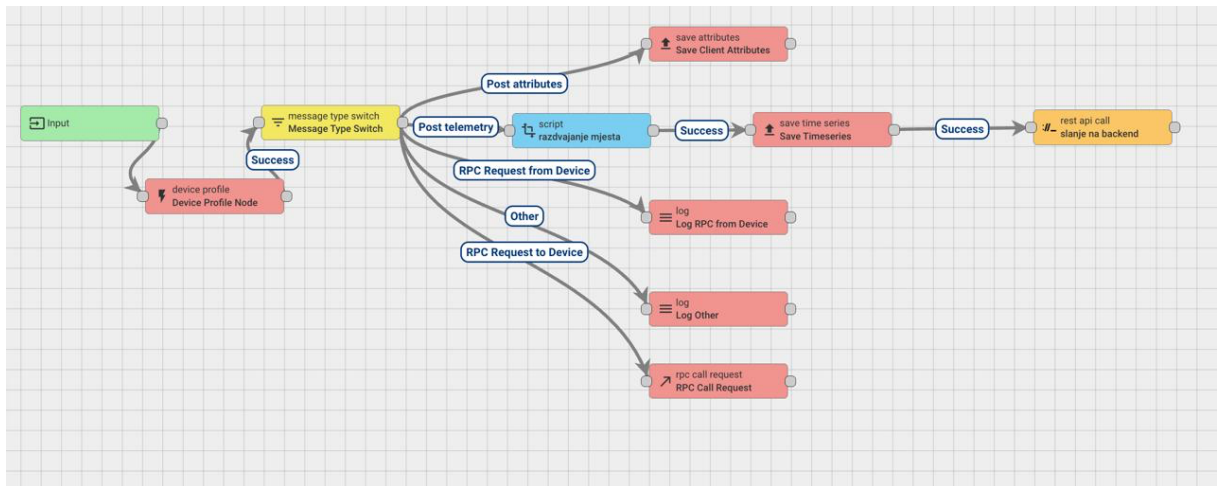
    msg.sim_mjesto = spot;

    msg.sim_stanje = occ;

    msg.tip_mjesta = "Simulirano mjesto";

}

return {msg: msg, metadata: metadata, msgType: msgType};
```



Nakon obrade u Script čvoru, podaci se prosleđuju u čvor Save Timeseries, čime se trajno zapisuju u bazu platforme i postaju dostupni za vizualizaciju.

3.3. Vizualizacija podataka (Dashboard)

Za potrebe nadzora razvijena je centralna nadzorna ploča unutar ThingsBoarda pod nazivom "SmartParking". Nadzorna ploča konfigurirana je za prikaz podataka u stvarnom vremenu (vremenski prozor: Realtime - last 1 minute), što omogućuje trenutnu reakciju sučelja na promjene u okolini.

Na nadzornoj ploči izrađeni su vizualni widgeti (HTML Value Cards) koji dinamički mijenjaju strukturu HTML-a i CSS stilove (boje) ovisno o vrijednostima telemetrije. Sustav jasno razlikuje i ispisuje o kojem se tipu parkirnog mjesta radi, kojoj garaži pripada te je li ono slobodno ili zauzeto.

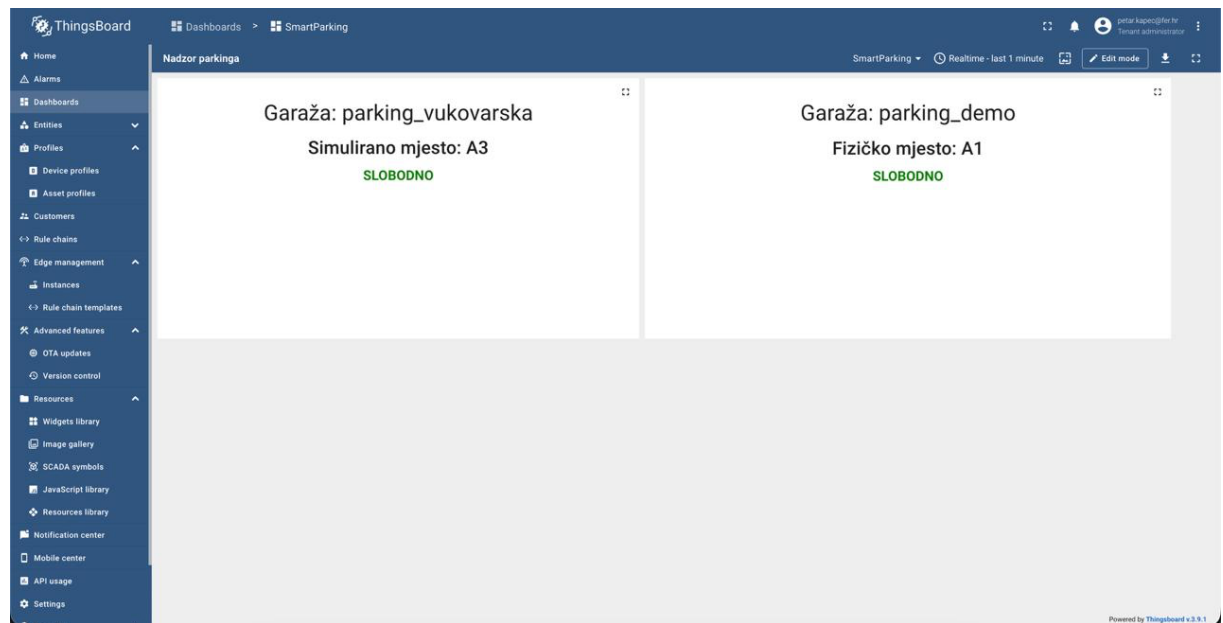
Primjerice, dok simulacijski modul vrti podatke, na sučelju se prikazuju informacije poput:

Garaža: parking_vukovarska

Simulirano mjesto: A2

SLOBODNO (prikazano zelenom bojom).

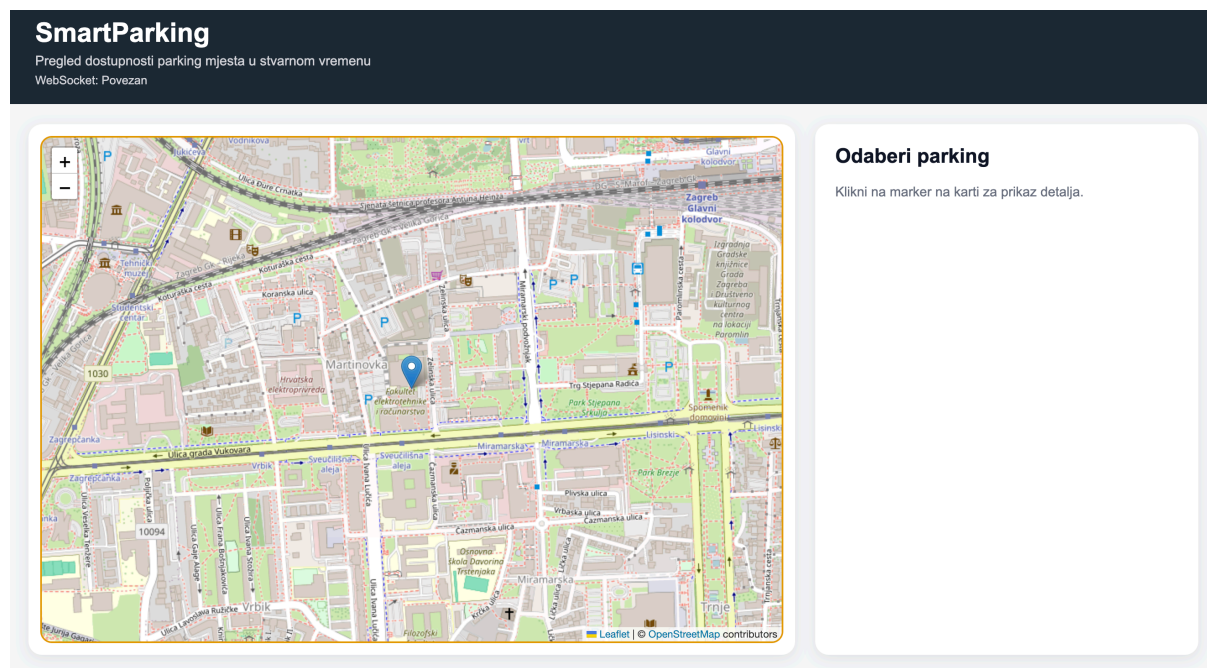
Kada fizički senzor detektira promjenu (npr. prekrivanje HC-SR04 senzora), nadzorna ploča trenutno prepoznaje prioritet senzora te se tekstualni i vizualni prikaz mijenjaju u podatke s lokacije parking_demo uz oznaku da se radi o fizičkom mjestu A1. Logika za izmjenu boja obavlja se lokalno na samoj kartici primjenom $f(x)$ funkcije koja boolean vrijednosti pretvara u odgovarajuće vizualne indikatore.



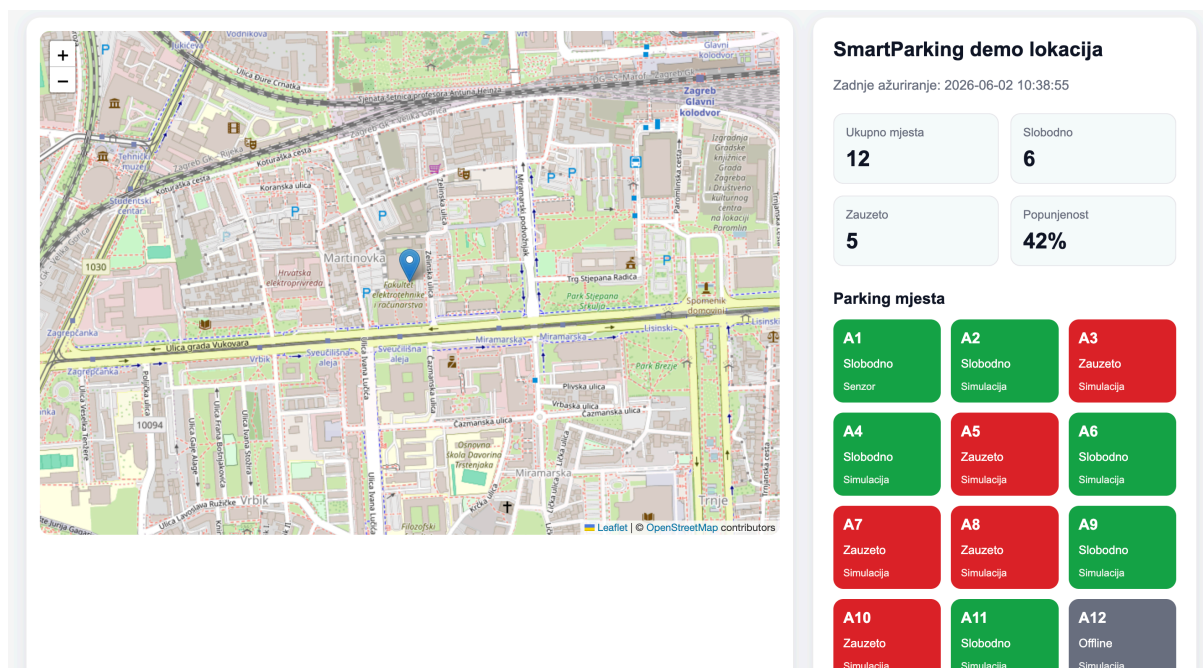
4. Korisničke aplikacije

U sklopu projekta razvijena je web aplikacija za prikaz zauzetosti parkirnih mjesta. Aplikacija korisniku omogućuje pregled lokacije parkinga na karti, odabir parkinga te prikaz trenutnog stanja pojedinih parkirnih mjesta.

Web aplikacija prikazuje parking pomoću OpenStreetMap karte. Klikom na marker parkinga otvara se detaljni prikaz s osnovnim informacijama o parkingu. U detaljnom prikazu prikazuje se ukupan broj parkirnih mjesta, broj slobodnih mjesta, broj zauzetih mjesta i postotak popunjenosti. Ispod osnovne statistike prikazana su pojedina parkirna mjesta.



Stanje svakog parkirnog mjesta prikazano je bojom. Zelena boja označava slobodno mjesto, crvena boja označava zauzeto mjesto, a siva boja označava mjesto za koje trenutno nema dostupnih podataka ili je označeno kao offline. Uz svako mjesto prikazuje se i izvor podatka, primjerice senzor ili simulacija.



Web aplikacija izrađena je pomoću Reacta, a za razvoj i izgradnju aplikacije koristi se Vite razvojno okruženje. Za prikaz karte koristi se Leaflet biblioteka zajedno s OpenStreetMap kartografskim podacima. Aplikacijski dio sustava dodatno je kontejneriziran pomoću Dockera, pa se frontend, backend, MQTT broker, simulacija, gateway i HAProxy pokreću kao zasebni servisi definirani u Docker Compose konfiguraciji.

Korisnik aplikaciji pristupa putem HAProxy servisa, koji predstavlja ulaznu točku u sustav. HAProxy prima HTTP zahtjeve na portu 80 i usmjerava ih prema odgovarajućem servisu. Zahtjevi za prikaz korisničkog sučelja prosljeđuju se frontend servisu, dok se API zahtjevi i WebSocket komunikacija prosljeđuju backend servisu. Na taj način korisnik pristupa aplikaciji preko jedne adrese, bez potrebe za poznavanjem internih portova pojedinih servisa.

Poslužiteljski dio aplikacije izrađen je pomoću Node.js i Express radnog okvira. Backend servis zadužen je za dohvat i ažuriranje podataka o parkiralištima i parkirnim mjestima. Za pohranu podataka koristi se SQLite baza podataka, koja se u Docker okruženju montira kao lokalna datoteka unutar backend containera. Time se osigurava da se stanje parkirališta ne izgubi ponovnim pokretanjem sustava.

Aplikacija pri pokretanju dohvaća trenutno stanje parkirališta putem REST API-ja. Nakon početnog dohвата uspostavlja se WebSocket veza s backendom. Kada poslužitelj primi promjenu stanja pojedinog parkirnog mjesta, primjerice od simulacije ili iz senzorskog dijela sustava, promjena se sprema u bazu podataka i odmah šalje web aplikaciji putem WebSocketsa. Na taj se način korisničko sučelje ažurira u stvarnom vremenu, bez potrebe za ručnim osvježavanjem stranice.

Najznačajniji implementacijski detalj web aplikacije je odvajanje prikaza od logike obrade podataka. React komponente služe za prikaz karte, statistike i pojedinačnih parkirnih mjesta, dok su dohvat podataka, WebSocket komunikacija i obrada promjena izdvojeni u zasebne servise i pomoćne funkcije. Takva organizacija olakšava održavanje

aplikacije i omogućuje jednostavnije proširenje sustava, primjerice dodavanjem novih parkirališta, dodatnih senzora ili novih izvora podataka.